

Chapter 4: System Design and Analysis

4.1 Introduction

This chapter serves as the definitive technical blueprint for the DropCity platform. It transitions from the high-level methodology of Chapter 3 into a low-level architectural specification. The design follows a **Reactive-Spatial Architecture**, optimized for the unique constraints of the Zimbabwean informal transport sector: specifically, high-latency networks, cash-based transactions, and a strict zero-detour mandate. This chapter provides sufficient detail for a full-stack developer to implement the database, matching engine, and security protocols.

4.2 Detailed Requirement Specification

4.2.1 Functional Requirements (FR)

- **FR1: Spatial Route Declaration:** Couriers must submit a "Declared Route" consisting of a Start Point, End Point, and a Google-encoded Polyline string. The system must convert this string into a PostGIS LineString for spatial indexing.
- **FR2: Zero-Detour Matching Engine:** The system must filter parcels based on a "dual-intersection" logic—both pickup and drop-off points must lie within a 500m spatial buffer of the courier's path.
- **FR3: Dynamic Virtual Interchange Assignment:** Upon a match, the system must calculate the coordinates on the road path closest to the client's location (ST_ClosestPoint) and designate these as "Virtual Bus Stops."
- **FR4: Three-Way Handshake (3WH):** A mandatory verification sequence requiring:
 1. **Spatial Verification:** Courier within 50m of the interchange.
 2. **Shared Secret:** Verification of a 6-digit OTP generated by the receiver.
 3. **Visual Proof:** Encrypted upload of "Before" and "After" parcel photos.
- **FR5: Resilient Data Synchronization:** An "Outbox Pattern" must be implemented to queue GPS heartbeats and transaction logs locally when the network signal drops below 10kbps.
- **FR6: Passenger-Proxy Pricing:** Automatic fee generation using the formula
$$F = \min((S \times D) + (P_r \times 2), 2)$$
- **FR7: Debt Ledger Management:** A tracking system for cash collected by couriers, preventing new matches once a "Commission Debt" threshold (e.g., \$10) is exceeded.

4.2.2 Non-Functional Requirements (NFR)

- **NFR1: Spatial Anonymity:** Courier GPS coordinates must be abstracted into "Progress Checkpoints" (e.g., "Passed Westgate") to ensure courier privacy.
- **NFR2: Computational Performance:** Matching queries must resolve in $< 800ms$ to ensure real-time responsiveness during high-traffic periods in the CBD.

- NFR3: Fault Tolerance:** The system must handle "Split-Brain" scenarios where Supabase (Source of Truth) and Firestore (Real-time Sync) temporarily diverge due to network partitioning.

4.3 System Modeling

4.3.1 Class Diagram (Core Entities)

To build this system, the developer must implement the following entity relationships:

- User:** (ID, Role[Courier/Client], Rating, DebtBalance)
- CourierProfile:** (UserID, VehicleType, LicensePlate, CurrentRouteID)
- Route:** (ID, Polyline, GeometryPath, StartTime, EndTime, Status)
- Parcel:** (ID, SenderID, ReceiverID, Size[S,M,L], Status, PickupInterchange, DropoffInterchange)
- Handshake:** (ParcelID, Step[Pickup/Dropoff], CourierLocation, ClientPIN, PhotoURL, Timestamp)

4.3.2 State Machine Diagram: Parcel Lifecycle

The system logic must strictly follow these transitions:

- PENDING:** Parcel created by Sender; no courier assigned.
- MATCHED:** Corridor matching algorithm pairs parcel with a Route.
- PICKUP_ZONE:** Courier enters 50m radius of Pickup Interchange.
- IN_TRANSIT:** Pickup Handshake complete (PIN + Photo uploaded).
- DROPOFF_ZONE:** Courier enters 50m radius of Drop-off Interchange.
- ARRIVED:** Courier at stop; Receiver notified to meet vehicle.
- DELIVERED:** Termination Handshake complete; Transaction closed.

4.3.3 Sequence Diagram: The Three-Way Handshake (Termination)

Step	Actor	Action	System Verification
1	Courier	Approaches I_d	Detects $ST_DWithin(Co$
2	System	Push Notification	Triggers "Courier is 2 minutes away" to Receiver
3	Receiver	Meets Vehicle	Requests "Release PIN" from App

4	Receiver	Verbal Handover	Gives 6-digit PIN to Courier
5	Courier	Input PIN	Validates PIN_Receiver == PIN_Entered
6	Courier	Final Photo	Captures Receiver holding the parcel
7	System	Finalize	Status → DELIVERED, Unlock Payout, Increment Debt

4.4 Logical Database Design

4.4.1 Relational Schema (Supabase/PostGIS)

The Relational DB handles the permanent "Source of Truth" and spatial calculations.

Table	Column	Type	Description
courier_routes	id	UUID (PK)	Unique Route ID
	route_path	GEOMETRY(LineString, 4326)	The road-accurate path (Indexed)
	corridor_buffer	GEOMETRY(Polygon, 4326)	Generated via ST_Buffer(route_path, 0.005)
parcels	id	UUID (PK)	Unique Parcel ID
	pickup_loc	GEOGRAPHY(Point)	Precise coordinate of sender
	dropoff_loc	GEOGRAPHY(Point)	Precise coordinate of receiver
	match_id	UUID (FK)	Links to courier_routes.id

4.4.2 Document Schema (Firestore/Firestore)

Firestore handles the "Ephemeral State" and Real-time UI updates.

Document: /active_deliveries/{parcelId}

```
{
  "status": "IN_TRANSIT",
  "progress": 0.65,
  "last_checkpoint": "Emerald Hill",
  "eta_minutes": 12,
  "courier_near_stop": false,
  "security": {
    "pickup_confirmed": true,
    "dropoff_pin_hash": "a665a45920422f9d..."
  }
}
```

4.5 Algorithmic Logic: The Zero-Detour Matcher

The matching engine must be implemented as a PostgreSQL Function (RPC) for maximum speed.

Pseudocode Logic:

1. **Input:** sender_pickup, receiver_dropoff, search_buffer (500m).
2. **Filter:** Select all routes where both points are within the buffer:
`SELECT route_id FROM courier_routes
WHERE ST_DWithin(route_path, sender_pickup, 0.005)
AND ST_DWithin(route_path, receiver_dropoff, 0.005);`
3. **Project:** For the best match, find the closest points on the line:
`SELECT ST_ClosestPoint(route_path, sender_pickup) AS virtual_pickup,
 ST_ClosestPoint(route_path, receiver_dropoff) AS virtual_dropoff;`
4. **Validate:** Ensure `ST_LineLocatePoint(route_path, virtual_pickup) < ST_LineLocatePoint(route_path, virtual_dropoff)` to confirm the courier is moving in the correct direction.

4.6 Security and Accountability Architecture

- **The PIN Protocol:** PINs are generated using a Cryptographically Secure Pseudo-Random Number Generator (CSPRNG). They are hashed (SHA-256) before being stored in the

database to prevent admin-side theft.

- **Driver Handover Logic:** If a driver change occurs, the CourierID in the active delivery document is updated via a signed QR-handshake between the outgoing and incoming drivers. The "Chain of Custody" table logs this transition.
- **The Debt Blockade:** A middleware check on every acceptOrder request:
IF (courier.debt > 10.00) REJECT with "Please settle commission debt to continue."

4.7 Interface and Experience Design

- **Courier Dashboard:** One-handed UI design. Large "Capture Photo" and "Enter PIN" buttons placed within thumb-reach to ensure the courier doesn't need to turn off the engine or park the vehicle.
- **Progress Visualization:** A linear stepper replaces the map for clients.
[PICKED UP] ---- [CBD] ---- [AVONDALE] ---- (YOU ARE HERE) ---- [UZ CAMPUS]
- **The Cash Receipt:** A digital receipt is generated for the courier to show the recipient, displaying the breakdown: Delivery: \$1.50, Size Surcharge: \$0.20, Total Due: \$1.70.

4.8 Conclusion

Chapter 4 provides the operational "Source of Truth" for the DropCity project. By defining the exact SQL spatial queries, the multi-party handshake sequence, and the dual-database synchronization strategy, this chapter ensures that the system is not only theoretically sound but practically buildable. The design prioritizes the courier's efficiency and the client's security, creating a scalable logistics framework for the Zimbabwean informal economy.